

Service Overview

ภาพรวมของบริการ (Service Overview)

PropertyDamageReport Service

Github repository <https://github.com/natchanonsarasang/PropertyDamageReportService>

1. Service Owner

นาย ณัฐชนน สาระสังข์ รหัสนักศึกษา 6609611907 ภาคปกติ

2. Service Purpose

PropertyDamageReport Service เป็นบริการที่รับผิดชอบการรับและจัดเก็บรายงานความเสียหายต่อทรัพย์สินจากประชาชนในระหว่างหรือหลังเหตุการณ์ภัยพิบัติ โดยทำหน้าที่เป็นจุดศูนย์กลางในการรวบรวมข้อมูลความเสียหาย เพื่อให้หน่วยงานรัฐสามารถตรวจสอบ จัดลำดับความสำคัญ และดำเนินการช่วยเหลือได้อย่างเป็นระบบ

3. Pain Point ที่แก้ไข

ในสถานการณ์ภัยพิบัติ ประชาชนมักไม่รู้ว่าจะรายงานความเสียหายที่ใด ข้อมูลกระจัดกระจาย และไม่สามารถติดตามสถานะการดำเนินการของหน่วยงานได้ ส่งผลให้การช่วยเหลือล่าช้า บริการนี้จึงถูกออกแบบมาเพื่อให้มีระบบรายงานความเสียหายที่เป็นมาตรฐาน ตรวจสอบได้ และติดตามผลได้

3. Target Users

ประชาชนผู้ได้รับผลกระทบจากภัยพิบัติ (Citizen)

ระบบของหน่วยงานรัฐที่มีหน้าที่รับผิดชอบเรื่องที่เกี่ยวข้อง

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
 - รับและจัดเก็บรายงานความเสียหายจากประชาชน
 - จัดการสถานะของรายงาน (workflow ของ report)
 - ตรวจจับรายงานซ้ำ (duplicate detection)
 - ส่งต่อรายงานไปยังหน่วยงานที่เกี่ยวข้อง
 - ให้ประชาชนสามารถติดตามสถานะรายงานของตนได้
 - รับคำร้องขอความช่วยเหลือฉุกเฉินแบบ optional พร้อมกับรายงานความเสียหาย และส่งต่อไปยัง Rescue Service ทันที
 - จัดการกรณีที่ Rescue Service ไม่ตอบสนอง โดย enqueue payload ไว้ใน SQS เพื่อ retry ภายหลัง โดยไม่กระทบต่อการรับรายงานหลัก
- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
 - การตรวจสอบข้อเท็จจริงของความเสียหายภาคสนาม
 - การประเมินมูลค่าความเสียหาย
 - การวางแผนหรือดำเนินการซ่อมแซม
 - การจัดสรรงบประมาณหรือการอนุมัติช่วยเหลือ
 - การติดตามขั้นตอนการดำเนินงานภายในของหน่วยงานหลังจากรับเรื่องแล้ว

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การรับหรือปฏิเสธรายงานที่ข้อมูลไม่ครบถ้วน
- การตรวจจับรายงานซ้ำภายในช่วงเวลาที่กำหนด
- การเปลี่ยนสถานะเมื่อมีการส่งต่อไปยังหน่วยงานที่เกี่ยวข้อง (เช่น forwarded)

การตัดสินใจอิงจาก:

- incident_id ของเหตุการณ์
- location ของรายงาน
- contact_phone ของผู้แจ้ง
- timestamp ของการส่งรายงาน
- business rules สำหรับ validation และ duplicate detection

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด โดยไม่ต้องรอรอนุมัติจากมนุษย์ในกรณีปกติ โดยบทบาทของบริการคือการรับเรื่อง ตรวจสอบเบื้องต้น และส่งต่อข้อมูลเท่านั้น ไม่เกี่ยวข้องกับการตัดสินใจหรือการดำเนินงานของหน่วยงานหลังจากรับเรื่องแล้ว

6. Owned Data

- Damage Report Master Data

ใช้เก็บข้อมูลหลักของรายงานความเสียหายที่ประชาชนส่งเข้ามา เช่น ประเภทความเสียหาย รายละเอียด สถานที่เกิดเหตุ ข้อมูลผู้แจ้ง ข้อมูลติดต่อ หลักฐานประกอบ และสถานะของรายงาน ข้อมูลชุดนี้เป็นข้อมูลหลักของบริการและถูกสร้างจากการใช้งานของระบบโดยตรง บริการนี้เป็นผู้กำหนด lifecycle ของรายงานตั้งแต่การสร้าง การส่งต่อ จนถึงการรับเรื่องจากหน่วยงานที่เกี่ยวข้อง จึงต้องควบคุมข้อมูลชุดนี้เองทั้งหมด

- AgencyResponses Data

ใช้เก็บข้อมูลการตอบกลับจากหน่วยงานที่ได้รับรายงานความเสียหาย โดยหนึ่งรายงานอาจได้รับการตอบกลับจากหลายหน่วยงาน ตารางนี้จึงถูกออกแบบเพื่อเก็บ response ของแต่ละหน่วยงานแยกจากกัน ข้อมูลที่จัดเก็บประกอบด้วยชื่อหน่วยงาน สถานะการตอบรับ และเหตุผลในการปฏิเสธในกรณีที่หน่วยงานไม่รับเรื่องรายงานนั้น

- Incident Data

ใช้เก็บข้อมูล incident ที่ synchronize มาจาก IncidentTracking Service ผ่าน scheduled Lambda process เพื่อใช้สำหรับอ้างอิงและตรวจสอบ incident ภายในระบบ ข้อมูลที่จัดเก็บประกอบด้วยประเภทเหตุการณ์ รายละเอียด สถานที่ ระดับความสำคัญ และสถานะของ incident โดยบริการนี้เป็นผู้ควบคุมข้อมูลที่ถูกระบุ synchronize และจัดเก็บไว้ใน DynamoDB สำหรับใช้งานภายในระบบเท่านั้น

7. Linked Data (Reference Only)

Agency / Organization Service

PropertyDamageReport Service มีการอ้างอิงข้อมูลหน่วยงานจาก Agency / Organization Service เพื่อใช้กำหนดปลายทางในการส่งต่อรายงานความเสียหายไปยังหน่วยงานที่เกี่ยวข้อง โดยบริการนี้จัดเก็บเพียงข้อมูลอ้างอิงที่จำเป็นต่อการทำงาน ได้แก่ agencyId และ agencyName เท่านั้น

บริการนี้ไม่ได้เป็นเจ้าของข้อมูลหน่วยงาน และไม่มีการจัดเก็บรายละเอียดเชิงลึกของหน่วยงาน เช่น ข้อมูลบุคลากร โครงสร้างองค์กร ช่องทางติดต่อภายใน หรือ workflow การดำเนินงานของหน่วยงาน ทั้งนี้เพื่อหลีกเลี่ยงความซ้ำซ้อนของข้อมูลและลด coupling ระหว่างบริการในระบบ distributed system

RescueRequest Service

PropertyDamageReport Service มีการเชื่อมต่อกับ RescueRequest Service เพื่อส่งต่อคำร้องขอความช่วยเหลือฉุกเฉิน ในกรณีที่ประชาชนระบุความต้องการความช่วยเหลือเร่งด่วนระหว่างการส่งรายงานความเสียหาย การเชื่อมต่อดำเนินการผ่าน API โดยบริการนี้จะจัดเก็บเฉพาะข้อมูลอ้างอิงที่จำเป็นสำหรับการติดตามผลการส่งต่อ ได้แก่ requestId, trackingCode และ rescueStatus เพื่อใช้ตรวจสอบสถานะการส่งคำร้องและรองรับกระบวนการ retry ในกรณีที่บริการปลายทางไม่ตอบสนอง บริการนี้ไม่ได้จัดเก็บข้อมูลภายในหรือ workflow ของ Rescue Service และไม่มีหน้าที่ในการบริหารจัดการกระบวนการช่วยเหลือ ซึ่งยังคงเป็นความรับผิดชอบของ Rescue Service โดยตรง

8. Non-Functional Requirements

- ระบบต้องไม่ crash เมื่อได้รับ request ที่ข้อมูลไม่ถูกต้อง และต้องตอบกลับด้วย error message ที่ชัดเจน
- ระบบต้องรองรับการส่งข้อมูลซ้ำแบบ **at-least-once delivery** ได้ โดยต้องมีการตรวจจับ duplicate report จาก incidentId, location และ contactPhone ภายในช่วงเวลาที่กำหนด
- API ต้องรองรับ **idempotency** เพื่อป้องกันการสร้างรายงานซ้ำจากการกดส่งหลายครั้งหรืออินเทอร์เน็ตไม่เสถียร
- ข้อมูลรายงานและข้อมูลติดต่อประชาชนต้องถูกจัดเก็บอย่างปลอดภัย และปฏิบัติตามหลัก **data privacy protection**
- หากบริการภายนอกที่ใช้ส่งต่อรายงานไม่ตอบสนอง ระบบต้องยังสามารถรับรายงานไว้ก่อนได้ (graceful degradation) และ retry การส่งต่อภายหลัง

Sync Contract

Synchronous Function Contract

Base URL: <https://k8wz4waf1f.execute-api.us-east-1.amazonaws.com/v1>

API Contract #1: Create Damage Report

ข้อมูลทั่วไป

- **Name:** Create damage report
- **Method:** POST
- **Path:** /damage-reports
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับให้ประชาชนส่งรายงานความเสียหายเข้าสู่ระบบ เมื่อบันทึกสำเร็จ ระบบจะสร้างข้อมูลใน Damage Report Master Data และหลังจากนั้นจะมีการ publish event แบบ asynchronous เพื่อส่งต่อหน่วยงานต่างๆที่เกี่ยวข้อง

Request

Path/Query Params

- ไม่มี

Headers

- Content-Type: application/json

Body:

```
{  
  
  "incidentId": "2FCBC2E0-5F71-44D1-A39B-15F0EDD5BB5C",  
  
  "damageType": "building",  
  
  "ownershipType": "personal",  
  
  "description": "บ้านพังเสียหายจากน้ำท่วม",  
  
  "location": "123 ถนนสีลม",  
  
  "reporterName": "สมชาย ใจดี",  
  
  "contactPhone": "0812345678",  
  
  "latitude": 13.7563,
```

```

"longitude": 100.5018,

"evidenceUrl": "https://img.com/1.jpg",

"rescueRequest": {                                // optional

  "requestType": "MEDICAL",

  "description": "ติดอยู่ชั้น 2 ออกไม่ได้",

  "locationDetails": "บ้านสีเหลืองข้างต้นไทร",

  "province": "กรุงเทพมหานคร",

  "district": "บางรัก",

  "subdistrict": "บางรัก",

  "addressLine": "123 ถนนสีลม"

}

}

```

Validation Rules

- incidentId : required, ต้องมีค่า
- damageType : required, อนุญาตเฉพาะ building, vehicle, infrastructure, utility, other
- ownershipType : required, อนุญาตเฉพาะ personal, public
- description : required, ต้องมีค่า
- location : required, ต้องมีค่า
- reporterName : required, ต้องมีค่า
- contactPhone : required, ต้องเป็นตัวเลข 10 หลักเท่านั้น เช่น 0812345678
- latitude : required, ต้องเป็นตัวเลข และมีค่าระหว่าง -90 ถึง 90
- longitude : required, ต้องเป็นตัวเลข และมีค่าระหว่าง -180 ถึง 180
- evidenceUrl : optional, ต้องเป็น URL ของรูปภาพหรือเอกสาร
- rescueRequest : optional, ข้อมูลคำขอความช่วยเหลือเพิ่มเติม

rescueRequest Fields (เมื่อส่ง rescueRequest มาด้วย)

- requestType : required, อนุญาตเฉพาะ MEDICAL, EVACUATION, SUPPLY
- description : required, ต้องมีค่า
- locationDetails : optional, รายละเอียดตำแหน่งเพิ่มเติม
- province : optional, ชื่อจังหวัด
- district : optional, ชื่ออำเภอหรือเขต
- subdistrict : optional, ชื่อตำบลหรือแขวง
- addressLine : optional, รายละเอียดที่อยู่เพิ่มเติม

Response

Success: 201 Created

```
{  
  
  "reportId": "REP-1747039245865",  
  
  "overallStatus": "forwarded",  
  
  "createdAt": "2026-05-17T10:00:00.000Z",  
  
  // เมื่อส่ง rescueRequest มาและสำเร็จ  
  
  "rescueRequest": {  
  
    "forwarded": true,  
  
    "requestId": "RES-001",  
  
    "trackingCode": "TRK-ABC",  
  
    "status": "pending",  
  
    "submittedAt": "2026-05-17T10:00:01.000Z"  
  
  },  
  
  // เมื่อส่ง rescueRequest มาแต่ RescueRequest service ล้มเหลว  
  
  "rescueRequest": {  
  
    "forwarded": false,  
  
    "error": "Rescue service returned 503",  
  
    "detail": { ... }  
  
  }  
  
}
```

Error: 400 Bad Request

```
{
```



```
"error": {  
  
  "code": "VALIDATION_ERROR",  
  
  "message": "contactPhone must be a 10-digit number",  
  
  "traceld": "uuid"  
}  
  
}
```

Dependency / Reliability

- rescue payload ที่ล้มเหลวถูก enqueue ไปยัง SQS retry queue พร้อม payload ครบถ้วน
- rescue service failure จะไม่กระทบ response หลัก report จะถูกบันทึกและ event ถูก publish เสมอ
- Validate ทุก field ก่อน ไม่มีการ save/publish จนกว่าจะผ่านทั้งหมด
- Timeout ≤ 30s

API Contract #2: Get Damage Report by ID

ข้อมูลทั่วไป

- **Name:** Get damage report detail

- Method: GET
- Path: /damage-reports/{reportId}
- Type: Synchronous

คำอธิบาย

ใช้ดึงรายละเอียดรายงานความเสียหายตาม reportId พร้อมสถานะปัจจุบันและข้อมูล RescueRequest (ถ้ามี)

Request

Path/Query Param

- reportId (string, required)

Headers

- Accept: application/json

Body

- ไม่มี

Response

Success: 200 OK

```
{  
  
  "reportId": "REP-1779039245865",  
  
  "incidentId": "2FCBC2E0-5F71-44D1-A39B-15F0EDD5BB5C",  
  
  "damageType": "building",  
  
  "ownershipType": "personal",  
  
  "description": "บ้านพังเสียหายจากน้ำท่วม",  
  
  "location": "123 ถนนสีลม",  
  
  "latitude": 60,  
  
  "longitude": 100.5018,  
  
  "reporterName": "นน ศรีบทส12",  
  
}
```

```
    "contactPhone": "0812345678",

    "evidenceUrl": null,

    "overallStatus": "forwarded",

    "assignedAgency": null,

    "rescueStatus": "failed",

    "rescueFailReason": "fetch failed",

    "rescueRequestId": null,

    "rescueTrackingCode": null,

    "rescueSubmittedAt": null,

    "createdAt": "2026-05-17T17:34:05.865Z",

    "updatedAt": "2026-05-17T17:34:08.166Z"
  }
}
```

Error: 400 Bad Request

```
{

  "error": {

    "code": "INVALID_ID_FORMAT",

    "message": "invalid reportId format",

    "traceId": "uuid"

  }

}
```

Error: 404 Not Found

```
{

  "error": {

    "code": "NOT_FOUND",

    "message": "Report not found",
```

```
"traceld": "uuid"  
}  
}
```

Dependency / Reliability

- เป็น read-only operation
- ไม่มี synchronous dependency กับ service อื่น
- ใช้ database read เพียงอย่างเดียว
- Timeout $\leq 30s$
- รองรับ retry ได้โดยไม่มี side effect

API Contract #3: List Damage Reports

ข้อมูลทั่วไป

- **Name:** List damage reports

- **Method:** GET
- **Path:** /damage-reports
- **Type:** Synchronous

คำอธิบาย

ดึงรายการรายงานความเสียหายทั้งหมด สามารถ filter ตาม overallStatus หรือ contactPhone ได้

Request

Path/Query Param

- contactPhone (optional, filter ตามเบอร์ผู้แจ้ง)
- overallStatus (optional, new / forwarded / acknowledged)

Headers

- Accept: application/json

Body

- ไม่มี

Response

Success: 200 OK

```
{  
  
  "items": [  
  
    {  
  
      "reportId": "REP-1779037773876",  
  
      "incidentId": "2FCBC2E0-5F71-44D1-A39B-15F0EDD5BB5C",  
  
      "damageType": "building",  
  
      "ownershipType": "personal",  
  
    }  
  ]  
}
```

```
    "description": "บ้านพังเสียหายจากน้ำท่วม",
    "location": "123 ถนนสีลม",
    "latitude": 13.7563,
    "longitude": 100.5018,
    "reporterName": "natchanon",
    "contactPhone": "0812345678",
    "evidenceUrl": null,
    "overallStatus": "forwarded",
    "assignedAgency": null,
    "rescueStatus": null,
    "rescueFailReason": null,
    "rescueRequestId": null,
    "rescueTrackingCode": null,
    "rescueSubmittedAt": null,
    "createdAt": "2026-05-17T17:09:33.876Z",
    "updatedAt": "2026-05-17T17:09:34.516Z"
  },
  {
    "reportId": "REP-1779039518607",
    "incidentId": "2FCBC2E0-5F71-44D1-A39B-15F0EDD5BB5C",
    "damageType": "building",
    "ownershipType": "personal",
    "description": "บ้านพังเสียหายจากน้ำท่วม",
    "location": "1234 ถนนสีลม",
    "latitude": 60,
```

```
{
  "longitude": 100.5018,
  "reporterName": "ณัฐชนน",
  "contactPhone": "0812345678",
  "evidenceUrl": null,
  "overallStatus": "forwarded",
  "assignedAgency": null,
  "rescueStatus": "failed",
  "rescueFailReason": "fetch failed",
  "rescueRequestId": null,
  "rescueTrackingCode": null,
  "rescueSubmittedAt": null,
  "createdAt": "2026-05-17T17:38:38.607Z",
  "updatedAt": "2026-05-17T17:38:40.861Z"
}
]
```

Dependency / Reliability

- เป็น read-only operation (idempotent โดยธรรมชาติ)
- ไม่เรียก service ภายนอก
- query จาก DynamoDB
- Timeout $\leq$ 30s

API Contract #4: Agency Response

ข้อมูลทั่วไป

- **Name:** Agency respond to damage report
- **Method:** POST
- **Path:** /agency-responses
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับให้หน่วยงานตอบรับหรือปฏิเสธรายงานความเสียหาย ถ้าหน่วยงานตอบ accepted ระบบจะ Update overallStatus เป็น acknowledge และ publish event เพื่อแจ้งให้หน่วยงานอื่นรู้ว่ารายงานนี้มีคนรับเรื่องแล้ว

Request

Path/Query Param

- ไม่มี

Headers

- Accept: application/json

Body

```
{  
  
  "reportId": "REP-1773151475212",  
  
  "agencyName": "PathumThaniLocalAuthority",  
  
  "status": "accepted"  
  
}
```

validation Rules

- **reportId** ต้องมีอยู่ใน DamageReports table
- **agencyName** ต้องไม่ว่าง
- **status** ต้องเป็น accepted หรือ rejected
- **rejectReasonCode** เป็น optional แต่ควรมีเมื่อ status = rejected

Response

Success: 200 OK Agency Service accept


```
{  
  
  "message": "Agency response recorded",  
  
  "responseId": "RESP-1773152000000",  
  
  "reportId": "REP-1773151475212",  
  
  "status": "accepted"  
  
}
```

Success: 200 OK Agency Service reject

```
{  
  
  "message": "Agency response recorded",  
  
  "responseId": "RESP-1773153000000",  
  
  "reportId": "REP-1773151475212",  
  
  "status": "rejected"  
  
}
```

ตัวอย่าง request ที่ปฏิเสธ

```
{  
  
  "reportId": "REP-1773151475212",  
  
  "agencyName": "DisasterManagementService",  
  
  "status": "rejected",  
  
  "rejectReasonCode": "OUT_OF_JURISDICTION"  
  
}
```

Error: 400 Bad Request

```
{  
  
  "error": {  
  
    "code": "VALIDATION_ERROR",
```

```
"message": "missing required field",
```

```
"traceId": "uuid"
```

```
}
```

```
}
```

Error: 400 Invalid Status

```
{
```

```
  "error": {
```

```
    "code": "INVALID_STATUS",
```

```
    "message": "status must be accepted or rejected",
```

```
    "traceId": "uuid"
```

```
  }
```

```
}
```

Dependency / Reliability

1. เป็น synchronous API
2. ไม่มี synchronous dependency กับ service อื่น
3. ใช้ DynamoDB write

API Contract #4: Get All Incident

ข้อมูลทั่วไป

- **Name:** List incidents

- Method: GET
- Path: /incidents
- Type: Synchronous

คำอธิบาย

ดึงรายการ incident ที่ระบบ synchronize มาจาก external Service

Request

Path/Query Param

- ไม่มี

Headers

- Accept: application/json

Body

ไม่มี

Response

Success: 200 OK

```
{  
  
  "items": [  
  
    {  
  
      "incidentId": "2FCBC2E0-5F71-44D1-A39B-15F0EDD5BB5C",  
  
      "incidentDescription": "Flood test",  
  
      "incidentType": "flood",  
  
      "location": "TU Dome's outdoor parking lot",  
  
      "priority": "High",  
  
      "status": "CLOSED"  
    }  
  ]  
}
```

}

Dependency / Reliability

- เป็น read-only operation (idempotent โดยธรรมชาติ)
- ไม่มี synchronous dependency กับ service ภายนอก
- query ข้อมูลจาก DynamoDB
- Timeout $\leq 30s$

Async Contract

Asynchronous Function Contract

Message Contract #1: DamageReportForwarded

ข้อมูลทั่วไป

- **Message Name:** DamageReportForwarded
- **Interaction Style:** Event-driven / Publish-Subscribe
- **Message Broker:** Amazon SNS Topic → Amazon SQS
- **Producer:** PropertyDamageReport Service
- **Consumer:** Agency Services (เช่น LocalAuthorityService, DisasterManagementService)
- **Channel/Topic:** damage-report.forwarded.v1
- **Version:** v1

คำอธิบาย

Event นี้จะถูก publish หลังจากระบบบันทึก Report ที่ถูกสร้างจากประชาชน เขียนลงฐานข้อมูลเรียบร้อยแล้ว โดย PropertyDamageReport Service จะส่ง event ไปยัง SNS Topic เพื่อกระจายข้อมูลรายงานความเสียหายไปยังหน่วยงานที่เกี่ยวข้องผ่าน SQS Queue แบบ asynchronous event-driven communication หลังจาก publish event สำเร็จ ระบบจะอัปเดตสถานะของรายงานในฐานข้อมูล เพื่อระบุว่ารายงานได้ถูกส่งต่อไปยังหน่วยงานแล้ว

Request (Event Publish)

Event message จะถูก publish ไปยัง SNS Topic ในรูปแบบ JSON ดังนี้

Message Body

```
{
  "eventType": "DamageReportForwarded",
  "eventId": "a3d9c3e2-42e1-4e6e-b0d1-7f0cfa55b2d1",
  "occurredAt": "2026-02-21T10:00:00Z",
  "data": {
    "reportId": "REP-001",
    "incidentId": "INC-2026-01",
    "damageType": "building",
    "ownershipType": "personal",
    "description": "Roof damaged after storm",
    "location": "Rangsit, Pathum Thani",
    "reporterName": "Somchai Jaidee",
    "contactPhone": "0812345678",
    "evidenceUrl": "https://img.com/1.jpg",
```

```
"createdAt": "2026-02-21T10:00:00Z"
}
}
```

Field Definition (Body)

Field	Type	Required	Description
eventType	string	Y	ชนิดของ event
eventId	UUID	Y	รหัส event
occurredAt	datetime	Y	เวลาที่ event ถูกสร้าง
data	object	Y	ข้อมูลของ damage report

Event Payload (data)

Field	Type	Required	Description
reportId	string	Y	รหัสรายงานความเสียหาย
incidentId	string	Y	รหัสเหตุการณ์ที่เกี่ยวข้อง
damageType	enum	Y	ประเภทความเสียหาย
ownershipType	enum	Y	ประเภทความเป็นเจ้าของทรัพย์สิน (personal / public)
location	string	Y	ตำแหน่งที่เกิดความเสียหาย
description	string	Y	รายละเอียดความเสียหาย
reporterName	string	Y	ชื่อผู้ยื่นคำร้อง
contactPhone	string	Y	เบอร์ติดต่อผู้แจ้ง
evidenceUrl	string	N	ลิงก์รูปหลักฐาน
createdAt	datetime	Y	เวลาที่สร้างรายงาน

Validation Rules

1. reportId ต้องมีอยู่จริงใน Master Data และไม่ว่าง
2. incidentId ต้องไม่ว่าง
3. location ต้องไม่ว่าง
4. eventId ต้องไม่ซ้ำ (idempotency check)

Response

Event นี้เป็น asynchronous communication จึงไม่มี synchronous response กลับมายัง producer หลังจาก publish event ไปยัง SNS Topic สำเร็จ ระบบจะถือว่าการส่งต่อรายงานเสร็จสิ้น และไม่รอผลลัพธ์จาก consumer services การตอบกลับจากหน่วยงานจะเกิดขึ้นผ่าน API POST /v1/agency-responses ซึ่งจะถูกจัดเก็บใน AgencyResponses table

Message Contract #2: DamageReportAcknowledged

ข้อมูลทั่วไป

- **Message Name:** DamageReportAcknowledged
- **Interaction Style:** Event-driven / Publish-Subscribe
- **Message Broker:** Amazon SNS Topic → Amazon SQS
- **Producer:** PropertyDamageReport Service

- **Consumer:** Agency Services (เช่น LocalAuthorityService, DisasterManagementService)
- **Channel/Topic:** damage-report.acknowledged.v1
- **Version:** v1

คำอธิบาย

Event นี้จะถูก publish เมื่อมีหน่วยงานใดหน่วยงานหนึ่ง ตอบรับรายงานความเสียหาย (accept) ผ่าน synchronous API POST /v1/agency-responses หลังจากระบบบันทึกข้อมูลการตอบรับลงใน AgencyResponses table และอัปเดตสถานะของรายงานใน DamageReports table ระบบจะ publish event นี้ไปยัง SNS Topic เพื่อแจ้งให้ consumer services ทุกตัวทราบว่า รายงานนี้มีหน่วยงานรับเรื่องแล้ว และไม่จำเป็นต้องพิจารณาเพิ่มเติม

Request Event

Event message ถูกส่งในรูปแบบ JSON payload ดังนี้

Message Body

```
{  
  
  "eventType": "DamageReportAcknowledged",  
  
  "eventId": "c1c9fdd1-0f21-4f03-9f3a-9c0c82c7d211",  
  
  "occurredAt": "2026-02-21T10:10:00Z",  
  
  "data": {  
  
    "reportId": "REP-001",  
  
    "incidentId": "INC-2026-01",  
  
    "assignedAgency": "DisasterManagementService",  
  
    "acknowledgedAt": "2026-02-21T10:10:00Z"  
  
  }  
}
```

Field Definition (Body)

Field	Type	Required	Description
eventType	string	Y	ชนิดของ event
eventId	UUID	Y	รหัส event
occurredAt	datetime	Y	เวลาที่ event ถูกสร้าง
data	object	Y	ข้อมูลของ damage report

Event Payload (data)

Field	Type	Required	Description
reportId	string	Y	รหัสรายงานความเสียหาย
incidentId	string	Y	รหัสเหตุการณ์ภัยพิบัติ
assignedAgency	string	Y	หน่วยงานที่ตอบรับคำขอ
acknowledgedAt	datetime	Y	เวลาที่หน่วยงานตอบรับ

Validation Rules (Response)

- 1 reportId ต้องมีอยู่จริงใน DamageReports table
- 2 assignedAgency ต้องไม่ว่าง
- 3 eventId ต้องไม่ซ้ำ (idempotency check)

Response

Event นี้เป็นการสื่อสารแบบ asynchronous event-driven communication จึงไม่มี synchronous response จาก consumer services Consumer services ที่ subscribe queue จะได้รับ event เพื่อให้ service ทราบว่า report นี้มี agency รับแล้วและสามารถ ยกเลิกการพิจารณาหรือ ignore message ที่เกี่ยวข้องกับ report นี้ได้

Service Data

Service Data

1) Damage Report Master Data (Owned by this service)

Field Name	Type	Required	Description	Example
reportId	string	Y (PK)	รหัสรายงานความเสียหาย	REP-001
incidentId	string	Y	รหัสเหตุการณ์ภัยพิบัติที่เกี่ยวข้อง	INC-2026-01
damageType	enum	Y	ประเภทความเสียหาย (building / vehicle / infrastructure / utility / other)	building
ownershipType	enum	Y	ประเภททรัพย์สิน (personal / public)	personal
description	string	Y	รายละเอียดความเสียหาย	Roof damaged after storm
location	string	Y	พื้นที่ที่เกิดเหตุ	Rangsit, Pathum Thani
reporterName	string	Y	ชื่อผู้ยื่นคำร้อง	Somchai Jaidee
contactPhone	string	Y	เบอร์ติดต่อผู้แจ้ง	0812345678
evidenceUrl	string	N	ลิงก์รูปหลักฐาน	https://img.com/1.jpg
overallStatus	enum	Y	new / forwarded / acknowledged	forwarded
assignedAgency	string	N	หน่วยงานที่รับเรื่อง	DisasterManagementService
createdAt	datetime	Y	เวลาที่สร้างรายงาน	2026-02-21T10:00:00Z
updatedAt	datetime	Y	เวลาที่แก้ไขล่าสุด	2026-02-21T10:02:00Z
latitude	number	Y	ละติจูดของสถานที่เกิดเหตุ	13.7563
longitude	number	Y	ลองจิจูดของสถานที่เกิดเหตุ	100.5018

rescueStatus	enum	N	สถานะการส่ง rescue request (pending / success / failed)	failed
rescueFailReason	string	N	เหตุผลที่ RescueRequest service ล้มเหลว	RescueRequest service returned 503
rescueRequestId	string	N	request id จาก RescueRequest service	RES-001
rescueSubmittedAt	string	N	tracking code จาก RescueRequest service	TRK-ABC
rescueSubmittedAt	datetime	N	เวลาที่ส่ง rescue สำเร็จ	2026-05-17T10:00:01Z

2) AgencyResponses Table

เก็บ การตอบกลับจาก agency เพราะหลาย agency อาจตอบกลับมา

Field Name	Type	Required	Description	Example
responseld	string	Y (PK)	รหัส response	RESP-001
reportId	string	Y	report ที่เกี่ยวข้อง	REP-001

agencyName	string	Y	ชื่อหน่วยงาน	DisasterManagementService
status	enum	Y	accepted / rejected	accepted
rejectReasonCode	string	N	เหตุผล reject	Out of scope
respondedAt	datetime	Y	เวลาที่หน่วยงานตอบ	2026-02-21T10:05:00Z
createdAt	datetime	Y	เวลาบันทึก record	2026-02-21T10:05:00Z

3) Incidents Table

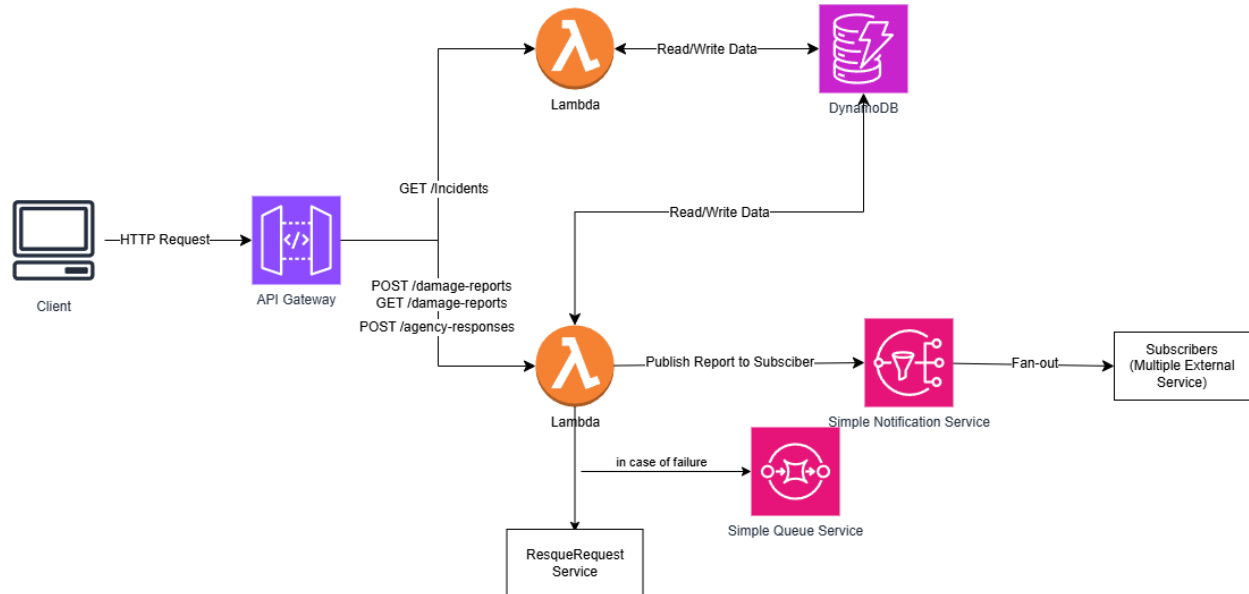
เก็บข้อมูล incident ที่ดึงมาจาก IncidentTracking service เพื่อใช้ในระบบ

Field Name	Type	Required	Description	Example
incidentId	string	Y (PK)	รหัส incident	2FCBC2E0-5F71-44D1-A39B-15F0EDD5BB5C

incidentType	string	N	ประเภทของ incident	Flood
incidentDescription	string	N	รายละเอียดของ incident	Heavy flooding reported in downtown area
location	string	N	รายละเอียดสถานที่เกิดเหตุ	TU Dome's outdoor parking lot
status	string	Y	สถานะของ incident	REPORTED
priority	string	Y	ระดับความสำคัญของ incident	High
syncedAt	datetime	Y	เวลาที่ระบบ sync ข้อมูลล่าสุด	2026-05-18T10:00:00Z

Service Architecture

Service Architecture



Component

- **Client:** Client คือผู้ใช้งานของระบบ ซึ่งประกอบด้วยประชาชนผู้ได้รับผลกระทบจากภัยพิบัติ และเจ้าหน้าที่หรือระบบภายนอกที่เกี่ยวข้อง โดยผู้ใช้งานสามารถส่ง HTTP request เพื่อสร้างรายงานความเสียหาย ติดตามสถานะรายงาน หรือดึงข้อมูลรายงานผ่าน API ของระบบ
- **Amazon API Gateway:** Amazon API Gateway ทำหน้าที่เป็น entry point ของระบบสำหรับรับคำขอแบบ synchronous จาก client และจัดการ routing ไปยัง AWS Lambda ที่เกี่ยวข้องกับ API Gateway รองรับ endpoint สำคัญของระบบ เช่น
 - **POST /damage-reports**
 - **GET /damage-reports**
 - **POST /agency-responses**
- **AWS Lambda:** AWS Lambda ทำหน้าที่ประมวลผล business logic หลักของ PropertyDamageReport Service โดยรับคำขอจาก Amazon API Gateway และดำเนินการตาม workflow ของระบบแบบ serverless หน้าที่สำคัญของ Lambda ได้แก่
 - ตรวจสอบความถูกต้องของข้อมูลรายงาน
 - บันทึกและดึงข้อมูลรายงานความเสียหายจาก DynamoDB
 - จัดการข้อมูลการตอบกลับจากหน่วยงาน
 - Publish event ไปยัง Amazon SNS แบบ asynchronous
 - ส่งต่อคำร้องขอความช่วยเหลือฉุกเฉินไปยัง RescueRequest Service ผ่าน API
 - จัดการ retry workflow และ enqueue message ลง Amazon SQS เมื่อ Rescue Service ไม่ตอบสนองหรือเกิด server error
 - ประมวลผล asynchronous task ภายในระบบ

- **Amazon DynamoDB** Amazon DynamoDB เป็นฐานข้อมูลหลักของระบบสำหรับจัดเก็บข้อมูลภายใน domain ของ PropertyDamageReport Service โดยแบ่งข้อมูลออกเป็นหลายตารางตามลักษณะการใช้งาน ดังนี้
 1. ข้อมูลรายงานความเสียหาย (DamageReports Table)
 2. ข้อมูลการตอบรับจากหน่วยงาน (AgencyResponses Table)
 3. ข้อมูลที่เกี่ยวข้องกับ Incident (Incident Table)
- **Amazon SNS (Simple Notification Service):** Amazon SNS ทำหน้าที่เป็นช่องทางสำหรับกระจายข้อมูลรายงานความเสียหายไปยัง subscriber หลายบริการแบบ asynchronous communication เมื่อมีการสร้างรายงานใหม่หรือเกิด event สำคัญ ระบบจะ publish event ไปยัง SNS topic เพื่อให้บริการอื่นสามารถ subscribe และรับข้อมูลไปประมวลผลต่อได้โดยไม่เกิด coupling โดยตรงระหว่างบริการ
- **Subscriber:** Subscriber คือบริการภายนอกหรือหน่วยงานที่ subscribe ข้อมูลจาก Amazon SNS เพื่อรับรายงานความเสียหายไปดำเนินการต่อ เช่น
 - LocalAuthority Service
 - Government Agency Service
 - Disaster Coordination Service

บริการเหล่านี้สามารถประมวลผลข้อมูลหรือดำเนินการช่วยเหลือตาม domain responsibility ของตนเองได้อย่างอิสระ

- **Amazon SQS:** Amazon SQS ถูกใช้เป็น message queue สำหรับจัดเก็บ rescue payload ที่ไม่สามารถส่งไปยัง Rescue Service ได้สำเร็จ เนื่องจากเกิด server error หรือปัญหาการเชื่อมต่อกับบริการปลายทาง เมื่อการส่งต่อไปยัง RescueRequest Service ล้มเหลว ระบบจะ enqueue payload ลงใน SQS เพื่อรอการ retry ภายหลัง

Explanation

Synchronous flow

ในส่วนของ synchronous flow เมื่อเกิดภัยพิบัติขึ้น เช่น น้ำท่วม ประชาชนหรือเจ้าหน้าที่จะส่งคำขอผ่าน Amazon API Gateway เช่น การสร้างรายงานความเสียหาย (POST /damage-reports) หรือการดึงข้อมูล (GET /damage-reports) คำขอจะถูกส่งต่อไปยัง AWS Lambda เพื่อประมวลผล

ระบบจะทำการตรวจสอบข้อมูล (validation) ทุก field ให้ผ่านก่อนทั้งหมด ได้แก่ ข้อมูล report หลัก เช่น damageType, ownershipType, contactPhone (ตัวเลข 10 หลัก), latitude/longitude รวมถึง rescueRequest (ถ้ามี) โดยระบบจะไม่บันทึกข้อมูลใดๆ จนกว่าจะผ่าน validation ทั้งหมด หลังจากนั้นจึงบันทึกข้อมูลลง DynamoDB และตอบกลับผู้ใช้งานทันที

ผู้ใช้สามารถแนบ rescueRequest มาพร้อมกับรายงานความเสียหายได้ (optional) เพื่อขอความช่วยเหลือฉุกเฉิน เช่น การแพทย์ (MEDICAL) การอพยพ (EVACUATION) หรือการส่งเสบียง (SUPPLY) โดยระบบจะดึง latitude, longitude, contactPhone และ reporterName จากข้อมูล report หลักโดยอัตโนมัติ ไม่จำเป็นต้องส่งซ้ำใน rescueRequest หลังจากบันทึก report และ publish event สำเร็จแล้ว Lambda จะ forward rescueRequest ไปยัง RescueRequest Service แบบ synchronous

หาก RescueRequest Service ตอบกลับด้วย server error (5xx) ระบบจะบันทึกสถานะ rescueStatus = failed ลงใน report และส่ง payload ที่ครบถ้วนพร้อม retry เข้า Amazon SQS โดยไม่กระทบต่อ response หลักที่ผู้ใช้ได้รับ

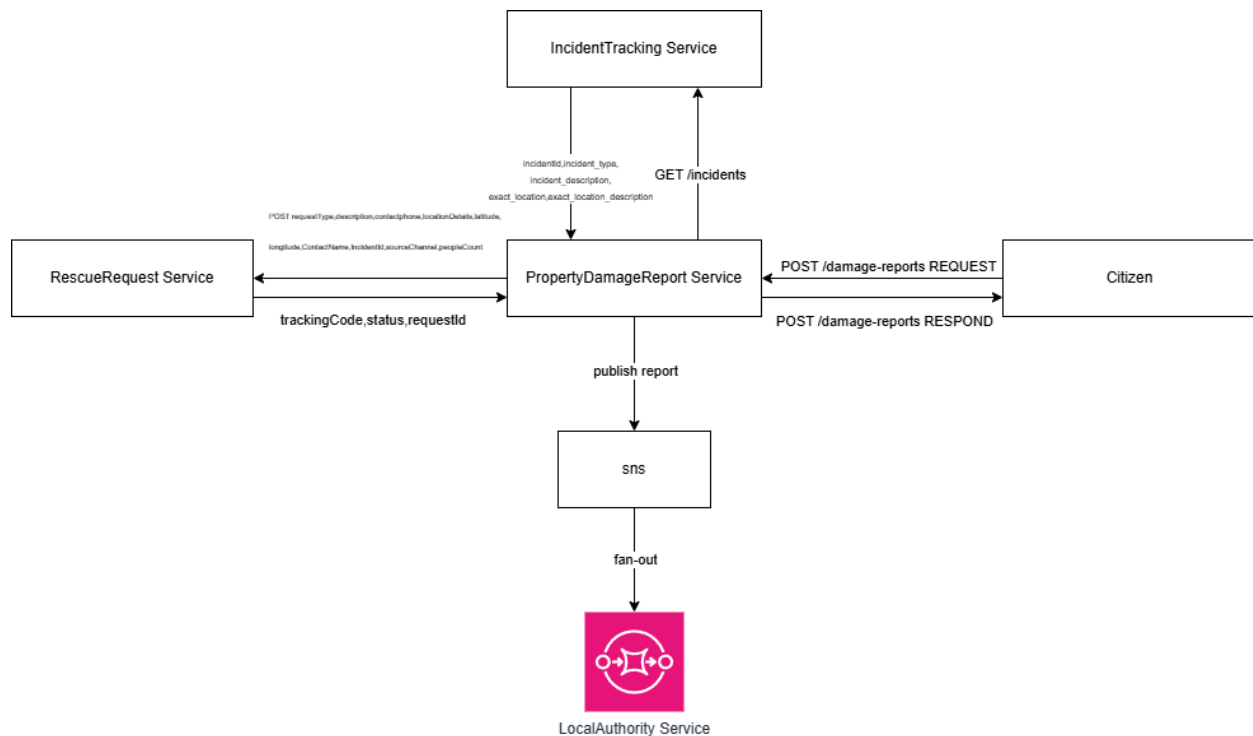
Asynchronous flow

ในส่วนของ asynchronous flow หลังจากทีประชาชนสร้างคำร้องความเสียหายสำเร็จ Lambda จะ publish event DamageReportForwarded ไปยัง Amazon SNS ซึ่งทำหน้าที่เป็นตัวกลางในการกระจาย event ไปยัง subscriber หลายตัว (fan-out) โดยแต่ละ subscriber ซึ่งเป็น service ภายนอกจะรับ event และนำไปประมวลผลต่อได้อย่างอิสระ เช่นการรับเรื่องโดยหน่วยงานที่เกี่ยวข้อง

นอกจากนี้ระบบยังรองรับการรับเรียกจาก service ภายนอกผ่าน endpoint POST /agency-responses โดย service ภายนอกสามารถแจ้งยืนยันการรับเรื่อง (accepted) หรือปฏิเสธ (rejected) พร้อม rejectReasonCode ได้ผ่าน API Gateway ซึ่งจะถูกประมวลผลโดย Lambda และบันทึกลง DynamoDB ในตาราง AgencyResponses แยกต่างหาก เมื่อ agency ตอบรับ ระบบจะอัปเดต overallStatus ของ report เป็น acknowledged และบันทึก assignedAgency ไว้สำหรับการติดตามสถานะต่อไป

Service Interaction

Service Interaction



Upstream Services (บริการต้นทางที่เรียกใช้งาน PropertyDamageReport Service)

1. Client (ประชาชน / Staff)

รูปแบบการสื่อสาร: Synchronous REST API

เรียกใช้งานผ่าน Amazon API Gateway โดยมี endpoint ดังนี้

- `GET /v1/incidents` - ดึงรายการ incident เพื่อให้ผู้ใช้เลือก incidentId ก่อนสร้างรายงาน
- `POST /v1/damage-reports` - สร้างรายงานความเสียหาย พร้อมแนบ rescueRequest (optional) เพื่อขอความช่วยเหลือฉุกเฉินได้ในคำขอเดียวกัน
- `GET /v1/damage-reports` - ดึงรายการรายงานทั้งหมด สามารถ filter ตาม overallStatus หรือ contactPhone
- `GET /v1/damage-reports/{reportId}` - ดึงรายละเอียดรายงานพร้อมสถานะปัจจุบัน รวมถึง rescueRequest status (ถ้ามี)

เมื่อมีการเรียกใช้งาน Lambda จะทำการ validate ทุก field ก่อนบันทึก ทั้ง report fields และ rescueRequest fields (ถ้ามี) โดยไม่มีการ save หรือ publish จนกว่าจะผ่าน validation ทั้งหมด

Downstream Services (บริการปลายทางที่ PropertyDamageReport Service เรียกหรือส่งข้อมูลไป)

1. IncidentTracking Service (Krittamet)

รูปแบบการสื่อสาร: Synchronous REST API

มี Lambda แยกต่างหากที่ทำหน้าที่ sync ข้อมูล incident จาก IncidentTracking Service มาเก็บใน DynamoDB ตาราง Incidents ของเรา โดยเรียก **GET /incidents** เพื่อดึงรายการเหตุการณ์ภัยพิบัติทั้งหมด ได้แก่ incidentId, incidentType, incidentDescription, location, status และ priority เพื่อให้ประชาชนสามารถเลือก incidentId เชื่อมโยงกับรายงานความเสียหายได้อย่างถูกต้อง หาก IncidentTracking Service ล่ม Lambda sync จะ fail แต่ข้อมูลเดิมใน Incidents table ยังคงใช้งานได้ตามปกติ

2. Rescue Service (Phattaraphum Kingchai)

รูปแบบการสื่อสาร: Synchronous REST API

เรียกใช้งานผ่าน **POST /rescue-requests** เมื่อผู้ใช้เขียน rescueRequest มาพร้อมกับรายงานความเสียหาย โดย forward ค่าขอหลังจาก validate และบันทึก report พร้อม publish event สำเร็จแล้วเท่านั้น รองรับ 3 ประเภท ได้แก่ MEDICAL (การแพทย์/ผู้ป่วยฉุกเฉิน), EVACUATION (อพยพออกจากพื้นที่/ติดค้าง) และ SUPPLY (อาหาร/น้ำดื่ม/เสบียง)

เมื่อเกิด 5xx / network error จะบันทึก rescueStatus = failed ลง DamageReports และ นำ payload ที่ครบถ้วนเข้า SQS Rescue Retry Queue เพื่อสำหรับ retry ในภายหลัง

Dependency Mapping

Dependency Mapping

Upstream Dependencies

1. Client (ประชาชน / Staff)

บทบาท: ผู้ใช้งานหลักของระบบ ทั้งประชาชนที่ได้รับผลกระทบจากภัยพิบัติและเจ้าหน้าที่ที่ต้องการดูข้อมูลรายงาน ส่ง HTTP request เข้ามาผ่าน Amazon API Gateway โดยตรง

Endpoints ที่เรียกใช้:

GET /v1/incidents ดึงรายการ incident เพื่อเลือก incidentId ก่อนสร้างรายงาน

POST /v1/damage-reports สร้างรายงานความเสียหาย พร้อมแนบ rescueRequest ได้ (optional)

GET /v1/damage-reports ดึงรายการรายงานทั้งหมด สามารถ filter ตาม status หรือ contactPhone

GET /v1/damage-reports/{reportId} ดึงรายละเอียดรายงานตาม reportId

รูปแบบ: Synchronous REST API ผ่าน Amazon API Gateway

ความสำคัญ: เป็น entry point หลักของระบบ ถ้า API Gateway ล่มจะไม่สามารถรับหรือดึงรายงานได้เลย

การจัดการเมื่อล่ม:

API Gateway ล่ม จะ ไม่มี fallback client จะได้รับ connection error ทันที ไม่มีทางเรียก Lambda ได้โดยตรง

ทุก error response จะมี traceId กลับไปเสมอ เพื่อให้ติดตาม log ใน CloudWatch ได้

Downstream Dependencies

1. RescueRequest Service (Phattaraphum)

บทบาท: รับคำร้องขอความช่วยเหลือฉุกเฉินที่ผู้ใช้แนบมาพร้อมกับรายงานความเสียหาย ระบบจะ forward ไปหลังจาก validate และบันทึก report สำเร็จแล้ว

Endpoint ที่เรียกใช้: POST {RESCUE_SERVICE_BASE_URL}/rescue-requests

รูปแบบ: Synchronous HTTP call

Trigger: เรียกเฉพาะเมื่อผู้ใช้แนบ rescueRequest object มาพร้อมกับ POST /v1/damage-reports เท่านั้น

ข้อมูลที่ส่งไป: incidentId, requestType (MEDICAL / EVACUATION / SUPPLY), description, peopleCount, latitude, longitude, contactName, contactPhone, sourceChannel, locationDetails, province, district, subdistrict, addressLine

ความสำคัญ: optional dependency เพราะ rescue flow แยกจาก report flow หลัก report ถูกบันทึกและ event ถูก publish ก่อนเสมอ ไม่ว่า rescue จะสำเร็จหรือไม่

การจัดการเมื่อล้ม:

5xx / network error จะบันทึก rescueStatus = failed ลง DamageReports และ นำ payload ที่ครบถ้วน เข้า SQS Rescue Retry Queue เพื่อ retry ที่หลัง report หลักยังถูกบันทึกและ response กลับ client ตามปกติ ไม่กระทบ flow หลัก

2. IncidentTracking Service (Krittamet)

บทบาท: ดึงข้อมูล incident จาก IncidentTracking Service มาเก็บใน DynamoDB ตาราง Incidents ของเรา เพื่อให้ประชาชนเลือก incidentId เชื่อมโยงกับรายงานความเสียหายได้

Endpoint ที่เรียก: GET {INCIDENT_SERVICE_BASE_URL}/incidents

รูปแบบ: Synchronous HTTP call

Trigger: Lambda นี้ถูกเรียกจากการตั้ง schedule ทุก 1 ชั่วโมง หรือเรียก manual เพื่อ sync ข้อมูล incident ล่าสุดมาเก็บไว้

ข้อมูลที่เก็บ: incidentId, incidentType, incidentDescription, location, status, priority ลง DynamoDB ตาราง Incidents

ความสำคัญ: ถ้า IncidentTracking Service ล่มจะไม่สามารถ sync ข้อมูลใหม่ได้ แต่ข้อมูลเดิมที่ sync ไว้แล้วยังใช้งานได้ตามปกติ ประชาชนยังเลือก incidentId ได้จากข้อมูลที่มีอยู่

การจัดการเมื่อล้ม:

IncidentTracking Service ล่ม จะทำให้ Lambda sync fail ทำให้ ข้อมูลใน Incidents table ไม่อัปเดต แต่ข้อมูลเดิมยังอยู่ครบ

ข้อมูล Incidents table เป็น read-only สำหรับ main Lambda ไม่กระทบ flow การสร้าง report

3. Amazon SQS (Rescue Retry Queue)

บทบาท: เก็บ rescue payload ที่ส่งไม่สำเร็จ (5xx) เพื่อ retry ที่หลัง

รูปแบบ: ส่งข้อความเข้าSQS เมื่อ rescue service ตอบ 5xx หรือ network error

Message payload ที่เก็บ: ครบทุก field ที่ rescue service ต้องการ ได้แก่ reportId, traceId, enqueuedAt และ rescuePayload (incidentId, requestType, description, peopleCount, latitude, longitude, contactName, contactPhone, sourceChannel, locationDetails, province, district, subdistrict, addressLine)

ความสำคัญ: ถ้า SQS ล่มตอน enqueue จะ log error ไว้แต่ไม่ throw

link

Github Link : <https://github.com/natchanonsarasang/PropertyDamageReportService>

Demo Video Link :

https://drive.google.com/file/d/1PpeGa8X3FJEN8rMHv5-NBO_0T0UsWvVg/view?usp=sharing